

VisionTrack - Computer Vision Object Detection System

Overview

VisionTrack is a Computer Vision project that detects and counts objects in images, videos, and live webcam streams. It uses OpenCV for image/video processing and a YOLO object-detection model for real-time detection.

The project demonstrates important Computer Vision concepts such as image acquisition, preprocessing, object detection, bounding boxes, confidence scores, object counting, and result visualization.

Problem Statement

Manual identification and counting of objects in images or video is time-consuming and difficult for large amounts of visual data. VisionTrack provides an automated system that processes visual input and identifies objects using a computer vision model.

Objectives

- Detect common objects automatically.
- Process images, videos, and webcam input.
- Display bounding boxes and confidence scores.
- Count detected objects by class.
- Save processed results for later analysis.
- Provide a modular and easy-to-test implementation.

Major Functional Modules

1. **Image Detection** - Detect objects in a single image.
2. **Video/Webcam Detection** - Detect objects frame-by-frame from video or a camera.
3. **Analytics & Reporting** - Count detected classes and save detection results.

Features

- Image object detection
- Video object detection
- Live webcam detection
- Bounding-box visualization
- Confidence score display
- Object counting
- CSV detection report
- Input validation and error handling
- Modular Python structure

Technologies Used

- Python 3.10+
- OpenCV
- Ultralytics YOLO
- NumPy
- Pandas
- PyTest

Project Structure

``text

Computer-Vision-Evaluated-Project/

```
|
├── README.md
└── statement.md
```

```

├── requirements.txt
├── .gitignore
├── config.py
├── main.py
├── src/
│   ├── __init__.py
│   ├── detector.py
│   ├── processor.py
│   ├── analytics.py
│   └── utils.py
├── tests/
│   ├── test_analytics.py
│   └── test_utils.py
├── docs/
│   ├── architecture.md
│   └── workflow.md
├── data/
│   ├── input/
│   └── output/
...

```

Installation

1. Clone the repository

```

```bash
git clone https://github.com/YOUR_USERNAME/visiontrack-computer-
vision.git
cd visiontrack-computer-vision
```

```

2. Create a virtual environment

```

```bash
python -m venv venv
```

```

Windows:

```

```bash
venv\Scripts\activate
```

```

Linux/macOS:

```

```bash
source venv/bin/activate
```

```

3. Install dependencies

```

```bash
pip install -r requirements.txt
```

```

The YOLO model file is downloaded automatically by Ultralytics on the first run.

How to Run

Image detection

```bash

python main.py --mode image --source data/input/sample.jpg

```

Video detection

```bash

python main.py --mode video --source data/input/sample.mp4

```

Webcam detection

```bash

python main.py --mode webcam

```

Press **Q** to stop webcam/video display.

Save a CSV report

```bash

python main.py --mode image --source data/input/sample.jpg --report

```

Input / Output

Input: JPG/PNG image, MP4/AVI video, or webcam stream.

Output: Processed image/video with bounding boxes. When `--report` is used, detection statistics are saved as CSV.

Testing

Run:

```bash

pytest

```

The tests validate utility functions and object-counting logic.

Computer Vision Concepts Demonstrated

- Image acquisition
- Image/video preprocessing
- Object detection
- Bounding boxes
- Confidence thresholding
- Frame-by-frame processing
- Object counting
- Result visualization
- Performance-aware processing

Evaluation Methodology

The project can be evaluated using:

- Detection accuracy/precision of the selected YOLO model.
- Number of correctly detected objects.
- False detections and missed detections.
- Processing speed/FPS for video or webcam.
- Correctness of generated object-count reports.

Non-Functional Requirements

- **Performance:** Process video frames efficiently.

- **Usability:** Simple command-line interface.
- **Reliability:** Validate input paths and handle invalid sources.
- **Maintainability:** Separate detection, processing, analytics, and utility modules.
- **Resource Efficiency:** Process frames sequentially instead of loading an entire video into memory.

Future Enhancements

- Custom training on a college-specific dataset.
- Object tracking across frames.
- Web dashboard for results.
- Database storage for detection history.
- Email/notification support.
- GPU performance optimization.

Academic Alignment

This project follows the VITyarthi Build Your Own Project requirements by providing multiple functional modules, a clear input/output workflow, modular source files, testing, architecture/workflow documentation, and Computer Vision model evaluation. The supplied guidelines require at least three major functional modules and clear input/output structure, plus 5-10 meaningful implementation files for coding projects.

[filecite\turn0file0\L24-L37](#) [filecite\turn0file0\L60-L63](#)

Author

Name: YOUR NAME

Registration No.: YOUR REGISTRATION NUMBER

Course: Computer Vision